payK Core — Project Reference & Interview Guide

High-Throughput Event-Driven Transaction Processing & Incentive Engine

Project Name: payK Core	**Developer:** Karan Suthar
Domain: FinTech / Payment Gateway	**Architecture:** Microservices & Kafka Streaming
Tech Stack: Java 17, Spring Boot, Kafka, JPA, H2	**Status:** Production Ready Portfolio Project

1. Executive Summary & Core Use Cases

payK Core is a high-performance backend microservice engineered to process real-time financial transfer events ingested from an Apache Kafka event stream. The engine enforces transactional integrity, validates sender balances to prevent overdrafts, fetches dynamic incentive rewards from an external REST service, and updates customer balances atomically while logging an immutable audit ledger.

Primary Real-World Use Cases:

- **Instant Peer-to-Peer (P2P) Payments:** Real-time balance transfer processing between accounts (similar to Venmo or Google Pay).
- **Promotional Rewards Engine:** Dynamically augmenting transfers with external cashback bonuses without delaying core transaction settlement.
- **Audit Ledger Maintenance:** Persisting immutable historical transaction records for financial compliance and balance query REST APIs.

2. System Architecture & Component Breakdown

The system follows an event-driven microservices pattern with clear separation of concerns across ingestion, business logic, external REST client integrations, and REST presentation layers.

Component Name	Class / File	Core Responsibility & Mechanism
Ingestion Consumer	`TransactionListener.java`	Listens to Kafka topic trader-updates under group payk-core-group. Deserializes JSON payloads into Transaction objects and forwards them to DatabaseConduit.
Transactional Engine	`DatabaseConduit.java`	Annotated with @Transactional. Validates sender balance (balance >= amount), fetches incentives, mutates balances, and persists TransactionRecord.

External REST Client	`IncentiveService.java`	Communicates with external API (http://localhost:8080/incentive) via RestTemplate. Implements fallback handling returning 0f on HTTP errors.
REST Query API	`BalanceController.java`	Exposes GET /balance?userId={id}. Retrieves real-time user balances from database and returns Balance DTO.
Database Entities	`UserRecord.java` `TransactionRecord.java`	JPA entities mapped to H2 tables. UserRecord tracks current balance; TransactionRecord maintains immutable audit history.

3. Uniqueness & Architectural Strengths

- **Asynchronous Decoupling via Kafka:** High-throughput transaction producers are completely decoupled from database writing, eliminating backpressure on payment originators.
- **Resilient Graceful Degradation:** If the external Incentive microservice experiences downtime or network latency, the core transaction settlement succeeds uninterrupted with a zero-incentive fallback.
- **ACID Transactional Guarantee:** Multi-user balance mutations and ledger insertion operate under Spring JPA transaction boundaries, preventing partial updates during failures.
- **Plug-and-Play Testability:** Uses Spring @EmbeddedKafka and Testcontainers for automated end-to-end integration testing without requiring external infrastructure.

4. Advantages & Trade-Off Analysis (Pros vs. Cons)

Key Advantages (Pros)	Disadvantages & Limitations (Cons)
✔ **High Ingestion Scale:** Handles massive event streams via Kafka partition scaling. ✔ **Atomic Consistency:** Database transfers execute within strict ACID transactions. ✔ **Fault Tolerance:** System gracefully handles external API outages. ✔ **Low Latency Reads:** Fast balance lookups over REST API.	✕ **Synchronous HTTP inside DB Lock:** Calling external API inside @Transactional holds database connection locks longer. ✕ **In-Memory DB Default:** H2 database is volatile (resets on restart). Production requires PostgreSQL/MySQL. ✕ **Concurrency Race Conditions:** Concurrent transfers on the same account require DB row locking (Pessimistic/Optimistic) to avoid lost updates.

5. Comprehensive Technical Interview Q&A; Guide

Essential questions interviewers will ask about payK Core and expert technical responses:

Q1: How does payK Core handle high concurrency and scaling using Kafka?

Kafka buffers incoming transfer events into topics partitioned across brokers. In payK Core, multiple instance nodes configured with the same consumer group (payk-core-group) automatically divide Kafka topic partitions among themselves. This achieves horizontal scaling for message ingestion while maintaining message ordering within individual partitions.

Q2: How does the system handle failures when calling the external Incentive microservice?

In IncentiveService.java, the call to restTemplate.postForObject() is wrapped in a try-catch block. If the external service is down, timed out, or returns a 5xx error, the service catches the exception and returns a default Incentive(0f) DTO. This ensures core transaction settlement is never blocked by optional promotional service failures.

Q3: Explain the transaction processing flow in DatabaseConduit. What happens if a sender has insufficient balance?

When processTransaction() is invoked, it queries UserRepository for the sender and recipient records. It evaluates the condition: sender != null && recipient != null && sender.getBalance() >= amount. If the sender's balance is lower than the transaction amount, the conditional check fails, no database mutation occurs, and no transaction record is created, preventing overdrafts.

Q4: What performance or locking issue exists in DatabaseConduit and how would you fix it in production?

Currently, incentiveService.fetchIncentive() is executed *inside* the @Transactional method processTransaction(). This means the database connection and locks are held open during the external HTTP round-trip. In production, I would refactor this by fetching the incentive *before* starting the DB transaction, or using a non-blocking reactive client like Spring's WebClient.

Q5: How would you prevent race conditions when two Kafka threads update the same user account simultaneously?

To prevent 'lost updates' or race conditions during concurrent transfers involving the same user, I would implement **Optimistic Locking** using a @Version field in UserRecord or **Pessimistic DB Row Locking** (using JPA @Lock(LockModeType.PESSIMISTIC_WRITE) in UserRepository). This ensures strict sequential update safety.

Q6: How would you make transaction processing idempotent if Kafka re-delivers duplicate messages?

To ensure idempotency, each incoming transaction would carry a unique transactionId or UUID key. Before processing, DatabaseConduit would check if a TransactionRecord with that ID already exists in the database. If present, the duplicate event is ignored.